第3回：音声信号とデジタル化

音声処理と認識

担当：小野原彩香
形式：動画回（非同期・自分のペースで）

📌 この回は動画回です。Google Formsでテストを受けてください。

本日の問いと学習のキーワード

> マイクが拾った音は、どのようにしてコンピュータが扱える数値の列になるのか？

今回のキーワード

- アナログ / デジタル
- サンプリング（標本化）・サンプリングレート F_s
- ナイキスト定理
- エイリアシング
- 量子化・ビット深度
- PCM（パルス符号変調）
- WAVファイル

音は「波」、コンピュータは「数値の列」

左図「アナログ信号」：なめらかに上下するサイン波（時間も振幅も連続）
右図「デジタル信号」：一定時間ごとにサンプルを取った点の集まり（離散値）

アナログ信号

- **連続的（Continuous）**：時間も振幅（値）も途切れなく変化する
- 例：マイクロフォンが空気振動を電気信号に変換した直後の状態

デジタル信号

- **離散的（Discrete）**：「ある瞬間の値」の有限個の数値の列
- 例：コンピュータのメモリに保存された音声データ

デジタル化は「近似」である。細かく・精密に数値化するほど音質が高くなる。

デジタル化 = サンプリング + 量子化

> アナログ信号（なめらかな波）→ [サンプリング] → 振幅値の時系列（連続値）→ [量子化] → 整数値の時系列（デジタルデータ）

ステップ1：サンプリング（標本化）

- アナログ信号を一定の時間間隔で値を取り出す
- 1秒間に何回サンプルを取るかを **サンプリングレート** F_s という（単位：Hz）
- 例：$F_s = 44100\ \mathrm{Hz}$ → 1秒間に44100回サンプル

ステップ2：量子化（Quantization）

- 各サンプルの振幅値を「有限個の整数値」に丸める
- 何段階の数値で表すかを **ビット深度** という
- 例：16 bit → $2^{16} = 65536$ 段階で振幅を表す

サンプリング — 時間を「飛び石」にする

上段：連続したサイン波（アナログ）
中段：等間隔の垂直な破線（サンプリングタイミング）が引かれた同じ波
下段：各タイミングでの波の値のみが点（●）として残っている

$$F_s\,(\mathrm{Hz}) = \frac{\text{サンプル数}}{1\ \text{秒}}$$

具体例

- $F_s = 44100\ \mathrm{Hz}$ のとき：1サンプルあたりの時間間隔

$$\frac{1}{44100} \approx 22.7\ \mu\mathrm{s}$$

サンプリングレートが低すぎると何が起きるか？ → 次のスライドのナイキスト定理で答える

ナイキスト定理

$$\boxed{F_s \geq 2 \times f_{\max}}$$

> サンプリングレート F_s は、記録したい最高周波数 $f_{\max}$ の **2倍以上** なければならない。

直感的な説明

高い音は波の山と谷の間隔が狭い（周期が短い）。
その波を正確に記録するには、少なくとも「1周期に2回以上」サンプルを取る必要がある。

> 上段「$F_s > 2f_{\max}$（十分なレート）」：元の波に近い形が復元される
> 下段「$F_s < 2f_{\max}$（不十分なレート）」：復元された波が元より低い周波数に見える（エイリアシング）

音声への適用例

- 電話（8 kHz サンプリング）のナイキスト周波数：4000 Hz
- → 4000 Hz 以上の成分は正確に記録できない

エイリアシング（Aliasing）

定義：ナイキスト周波数（$= F_s/2$）を超える周波数成分が「折り返して」低周波数の偽信号として現れる現象

具体例：$F_s = 8000\ \mathrm{Hz}$（ナイキスト周波数 = 4000 Hz）で 5000 Hz の信号を録音

$$5000\ \mathrm{Hz} - 8000\ \mathrm{Hz} = -3000\ \mathrm{Hz} \ \Rightarrow\ \text{折り返し：}3000\ \mathrm{Hz}\ \text{の偽信号が現れる}$$

対策：アンチエイリアシングフィルタ
サンプリング前にローパスフィルタでナイキスト周波数以上の成分を除去する。
デジタルオーディオ機器にはすべてこの処理が組み込まれている。

> 日常の例：映画で車輪が「逆回転」して見える現象もエイリアシングの一種（フレームレート < 車輪の回転周波数 × 2）

量子化 — 振幅を「段階」に変換する

上段：連続した正弦波
下段：同じ波に対して縦軸が「階段」状になっている（段階数が少ないと元の波からずれる）

ビット深度	段階数	用途
8 bit	$2^8 = 256$ 段階	古い通話音質・レトロゲームサウンド
16 bit	$2^{16} = 65{,}536$ 段階	CD品質・音声処理の標準
24 bit	$2^{24} = 16{,}777{,}216$ 段階	スタジオ録音・高品質音声
32 bit（float）	浮動小数点数	Python内部処理の標準

量子化ノイズ：ビット深度が低いほど丸め誤差が大きく「ザーッ」というノイズとして聴こえる

PCM — デジタル音声の最も基本的な形式

PCM（Pulse Code Modulation：パルス符号変調）

- サンプリング + 量子化で得られたデジタル音声データの表現形式
- 非圧縮（無損失）：元の信号情報がそのまま保存される
- MP3・AACなどはPCMデータを圧縮したもの

データサイズの計算

$$\text{ファイルサイズ（byte/秒）} = \frac{F_s \times \text{ビット深度} \times \text{チャンネル数}}{8}$$

例：CD音質（44100 Hz × 16 bit × 2ch）

$$= \frac{44100 \times 16 \times 2}{8} = 176{,}400 \text{ byte/秒} \approx 10.5 \text{ MB/分}$$

WAVファイルの中身を覗いてみる

WAVファイル = ヘッダ（身分証明書）＋ PCMデータ（実際の音声）

```
[RIFFヘッダ]（12 byte）
  ├── "RIFF" マジックナンバー
  ├── ファイルサイズ（4 byte）
  └── "WAVE"

[fmtチャンク]（24〜26 byte）
  ├── 音声フォーマット（PCM = 1）
  ├── チャンネル数（1=モノラル, 2=ステレオ）
  ├── サンプリングレート（Hz）
  └── ビット深度（8, 16, 24, 32）

[dataチャンク]（可変長）
  └── PCMサンプルデータ（実際の波形）
```

サンプリングレートごとの用途

サンプリングレート	ナイキスト周波数	用途
8,000 Hz	4,000 Hz	固定電話・VoIP・ウェイクワード検出
16,000 Hz	8,000 Hz	音声認識（Siri・Google・Whisper等）の標準
22,050 Hz	11,025 Hz	低品質MP3・ゲームサウンド
44,100 Hz	22,050 Hz	CD（人間の可聴域〜20,000 Hz を十分カバー）
48,000 Hz	24,000 Hz	動画・放送・YouTube

なぜCDは44.1 kHz か？

- 人間の可聴域：20 Hz〜20,000 Hz（加齢で高音域は低下）
- ナイキスト定理：$20{,}000 \times 2 = 40{,}000\ \mathrm{Hz}$ 以上が必要
- アンチエイリアシングフィルタのマージン＋歴史的経緯（NTSC規格のビデオテープ）で 44,100 Hz に

Python で WAVファイルを読む（1）— wave モジュール

```python
import wave
import numpy as np

filename = "sample.wav"

with wave.open(filename, 'r') as wf:
    n_channels  = wf.getnchannels()    # チャンネル数（1=モノラル, 2=ステレオ）
    sample_width = wf.getsampwidth()   # バイト数（2 = 16bit）
    frame_rate  = wf.getframerate()    # サンプリングレート（Hz）
    n_frames    = wf.getnframes()      # 総サンプル数
    duration    = n_frames / frame_rate

    print(f"チャンネル数: {n_channels}")
    print(f"ビット深度: {sample_width * 8} bit")
    print(f"サンプリングレート: {frame_rate} Hz")
    print(f"音声の長さ: {duration:.3f} 秒")

    raw_data = wf.readframes(n_frames)

audio_data = np.frombuffer(raw_data, dtype=np.int16)
```

確認ポイント：「波形データの形状 (32000,)」 = 2秒 × 16000 Hz = 32000サンプル

Python で WAVファイルを読む（2）— librosa で可視化

```
import librosa
import librosa.display
import matplotlib.pyplot as plt
from IPython.display import Audio

filename = "sample.wav"

# sr=None で元のサンプリングレートを保持（デフォルトでは22050Hzにリサンプリングされる）
y, sr = librosa.load(filename, sr=None)

print(f"サンプリングレート: {sr} Hz")
print(f"音声の長さ: {len(y) / sr:.3f} 秒")

# 波形を可視化
fig, ax = plt.subplots(figsize=(12, 4))
time_axis = np.arange(len(y)) / sr
ax.plot(time_axis, y, color='steelblue', linewidth=0.5)
ax.set_xlabel("時間（秒）")
ax.set_ylabel("振幅")
ax.set_title(f"音声波形（{filename}）")
plt.tight_layout(); plt.show()

# Colab上で音声を再生
Audio(y, rate=sr)
```

サンプリングレートを変えると音はどう変わるか

```
import librosa
import numpy as np
from IPython.display import Audio, display

y_original, sr_original = librosa.load("sample.wav", sr=None)
print(f"元のサンプリングレート: {sr_original} Hz")

# 異なるサンプリングレートにリサンプリングして耳で比較
for target_sr in [8000, 16000, 22050, 44100]:
    y_resampled = librosa.resample(y_original,
                                   orig_sr=sr_original,
                                   target_sr=target_sr)
    print(f"\n--- {target_sr} Hz ---")
    display(Audio(y_resampled, rate=target_sr))
```

体験の目的：耳で聴いて「8 kHz ではこもった音になる」「16 kHz で声の了解度は十分確保できる」という感覚をつかむ

よくある誤解と注意点

誤解1：「サンプリングレートが高ければ何でもよい」
→ 高いほどファイルサイズ・処理時間が増える。音声認識では 16 kHz で十分なことが多い。

誤解2：「ナイキスト定理は理論上の話」
→ 実際のシステムに必ず組み込まれている。アンチエイリアシングフィルタを入れないと認識精度が低下する。

誤解3：「librosa.load は wav しか読めない」
→ mp3・flac・ogg なども読める（内部でffmpegを使用）。ただし wav が最も確実。

誤解4：「16bit と 32bit(float) は同じデータ」
→ 16bit int は $-32768 \sim 32767$、32bit float は $-1.0 \sim 1.0$。Python で処理する際は 32bit float に変換する方が安定する。librosa はデフォルトで 32bit float で読み込む。

次回（第4回）の予告

形式：オンライン同期回（Zoom）
テーマ：音声の「周波数の時間変化」を可視化する
内容：フーリエ変換の直感的理解・短時間フーリエ変換（STFT）・スペクトログラムの読み方

事前学習（取り組んでおくと第4回の理解度が大幅に上がります）

1. このスライドのコード例を Google Colab に貼り付けて、自分の声や手持ちの音楽ファイルで実行してみる
2. 「フーリエ変換 わかりやすく」などで検索し、動画や図解を1つ見る（数式の理解は不要）

考えておく問い（任意・次回冒頭で取り上げます）

- 「ア」と「イ」の音は、スペクトログラムでどんな違いがあると思うか？

第3回のまとめ

概念	要点
アナログ vs デジタル	連続値 vs 離散値の数値列
サンプリング	一定時間ごとに値を取り出す。単位は Hz
ナイキスト定理	$F_s \geq 2 \times f_{\max}$ を守る必要がある
量子化	振幅を有限段階の整数に丸める。ビット深度が精度を決める
エイリアシング	ナイキスト超えの周波数が「偽物の低周波」として現れる現象
PCM	非圧縮のデジタル音声データの基本表現
WAVファイル	ヘッダ（メタ情報）＋ PCMデータからなるファイル形式

テスト：この回の後に Google Forms（5問・4択）に回答してください
締め切り：翌週末 23:59